GUÍA · EN / ES

Git Visual Lab

Guía visual y práctica de Git

Git es una máquina del tiempo para tu proyecto. Guarda fotos (commits), te deja trabajar ideas en paralelo (ramas) y combinarlas con seguridad (merge y rebase). Esta guía sigue los 10 niveles de la app interactiva: cada uno tiene una lección corta, el grafo inicial, los comandos que lo resuelven y el resultado.

Juégalo en línea: `davidcostacv.github.io/git-visual-lab/`

Cómo leer los grafos: los círculos son commits (los viejos a la izquierda). Las etiquetas redondeadas son ramas; la rellena es donde está HEAD. Las punteadas son tags. Los commits apagados ya no se alcanzan.

NIVEL 1 / 10

Commits: puntos de guardado

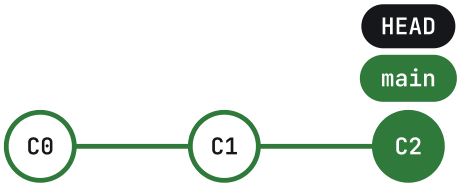
Un **commit** es una foto de todo tu proyecto en un momento, como un punto de guardado en un videojuego. Cada commit nuevo recuerda al anterior (su *padre*), así que juntos forman una historia a la que siempre puedes volver.

Cada círculo del grafo es un commit. **main** es el nombre de tu rama y **HEAD** marca dónde estás parado ahora.

ANTES



DESPUÉS



COMANDOS

```
$ git commit -m "Agregar página de inicio"
$ git commit -m "Agregar formulario de contacto"
```

En la vida real: En un proyecto real primero corres `git add archivo` para elegir qué entra en la foto y luego `git commit -m "qué hice"`. Un buen mensaje explica el *porqué*.

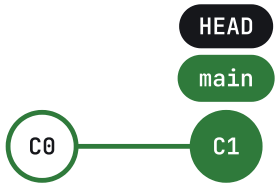
NIVEL 2 / 10

Ramas: mundos paralelos

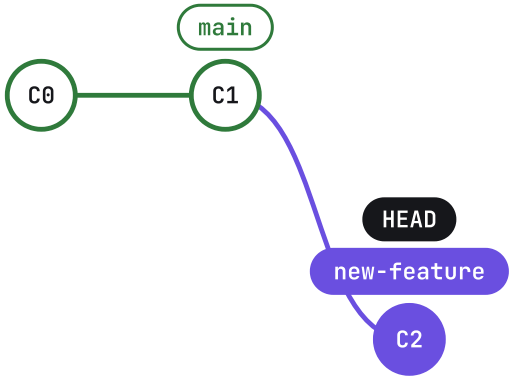
Una **rama** es solo una notita con un nombre, pegada a un commit. Crear una es instantáneo y no cuesta nada.

Las ramas te dejan probar una idea sin tocar **main**. Cuando haces commit, solo avanza la rama en la que estás (a la que está pegado HEAD).

ANTES



DESPUÉS



COMANDOS

```
$ git switch -c new-feature
$ git commit
```

En la vida real: Los equipos crean una rama por función o arreglo, por ejemplo `feature/login`. `git switch -c nombre` crea la rama y te mueve a ella en un solo paso.

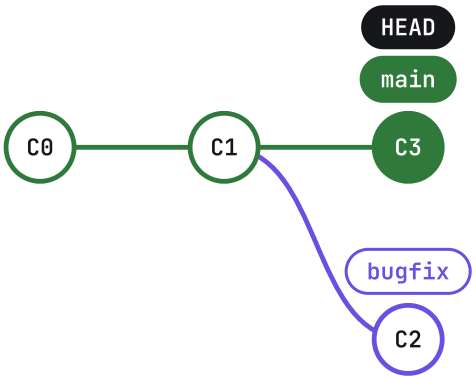
NIVEL 3 / 10

Merge: juntar el trabajo

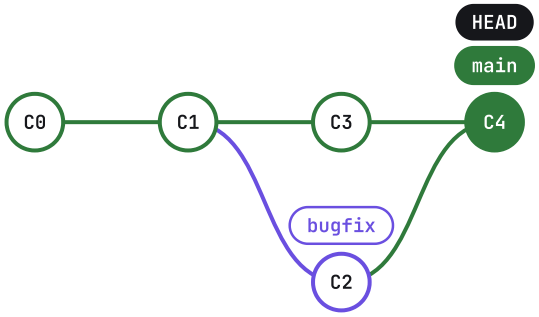
Tu arreglo vive en **bugfix**, pero los usuarios solo reciben lo que está en **main**. **git merge** une dos historias.

Como las dos ramas avanzaron, Git crea un **commit de merge**: un commit con *dos padres*. Ahora main contiene todo lo de ambos lados.

ANTES



DESPUÉS



COMANDOS

```
$ git merge bugfix
```

En la vida real: Párate en la rama que *recibe* el trabajo (casi siempre main) y haz merge de la otra. En GitHub, el botón "Merge pull request" hace exactamente esto.

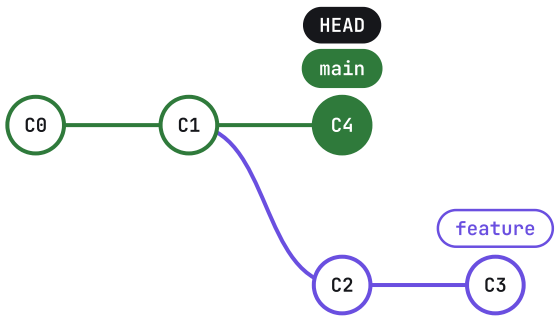
NIVEL 4 / 10

Rebase: historia en línea recta

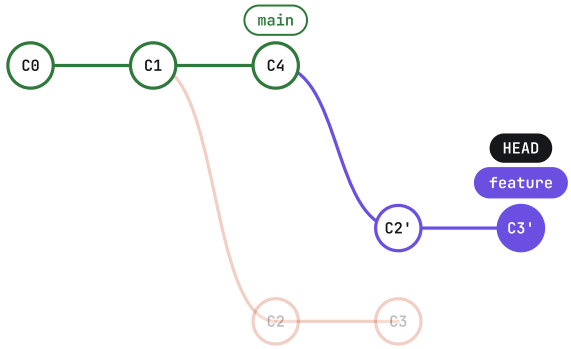
Rebase es la otra forma de combinar trabajo. En vez de un commit de merge, *copia* tus commits y los repite encima de otra rama, como si hubieras empezado tu trabajo hoy.

Las copias llevan una comilla: **C2** pasa a ser **C2'**. Los viejos se apagan porque ya nada los apunta.

ANTES



DESPUÉS



COMANDOS

```
$ git switch feature
$ git rebase main
```

En la vida real: Usa rebase para actualizar *tu propia* rama antes de abrir un pull request. Regla de oro: nunca hagas rebase de commits que otras personas ya bajaron, porque reescribe la historia.

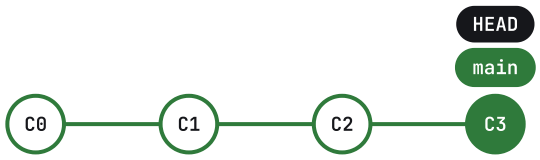
NIVEL 5 / 10

HEAD: dónde estás parado

HEAD significa "estás aquí". Normalmente está pegado a una rama. Si haces checkout de un *commit* en vez de una rama, HEAD queda **separado** (detached): estás visitando el pasado.

Cada commit tiene un id (C0, C1…). En Git real se ve como `a1b2c3d`.

ANTES



DESPUÉS



COMANDOS

```
$ git checkout C1
```

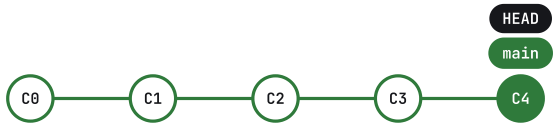
En la vida real: HEAD separado sirve para mirar código viejo o encontrar cuándo apareció un bug. Si quieres guardar lo que hagas ahí, crea una rama: `git switch -c rescate`.

NIVEL 6 / 10

Referencias relativas: ~ y ^

Escribir ids es lento. Git te deja moverte relativo a un nombre: **main^** es "el padre de main" y **main~3** es "3 pasos atrás de main". **HEAD~1** es "un paso atrás de donde estoy".

ANTES



DESPUÉS



COMANDOS

```
$ git branch review main~2
$ git checkout main^
```

En la vida real: `git diff HEAD~1` te muestra qué cambió en el último commit. `git branch -f nombre ref` mueve una rama a cualquier lado, útil para arreglar una rama creada en el lugar equivocado.

NIVEL 7 / 10

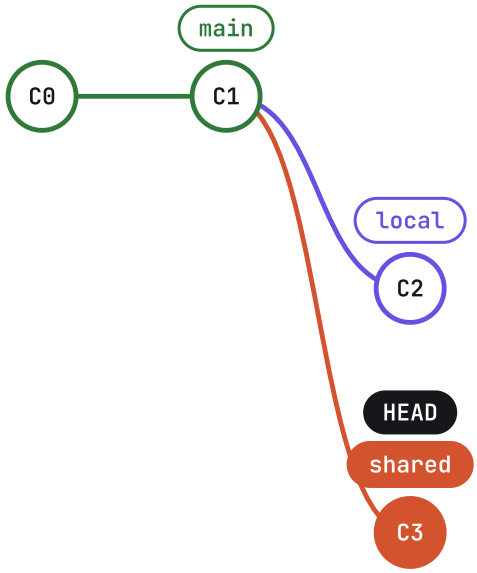
Deshacer: reset vs revert

Dos formas de deshacer un mal commit:

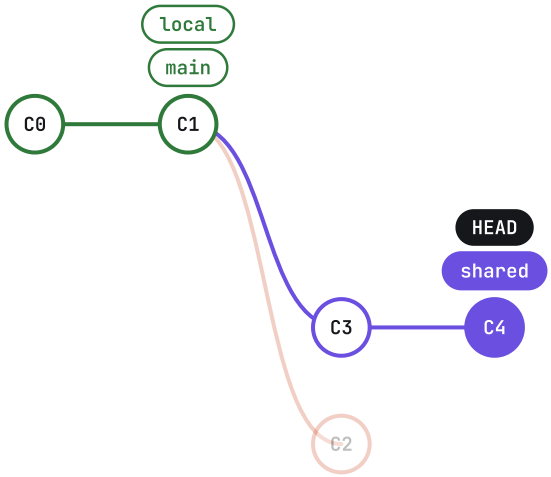
git reset mueve la rama hacia atrás, como si el commit nunca hubiera pasado. Úsalo solo en trabajo *privado*.

git revert agrega un commit *nuevo* que cancela al malo. La historia queda honesta, así que es seguro en ramas *compartidas*.

ANTES



DESPUÉS



COMANDOS

```
$ git switch local
$ git reset HEAD~1
$ git switch shared
$ git revert HEAD
```

En la vida real: Regla fácil: ¿ya hiciste push? Revert. ¿Solo está en tu PC? Reset está bien. En Git real reset tiene modos: `--soft` guarda tus cambios, `--hard` los bota.

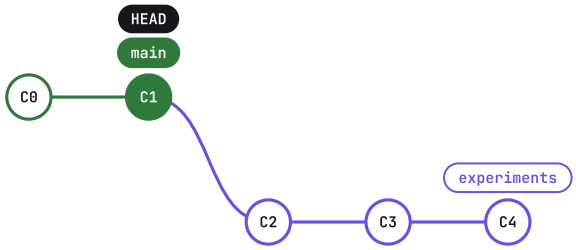
NIVEL 8 / 10

Cherry-pick: escoger commits

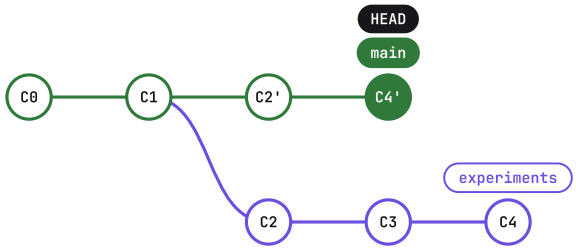
A veces quieres *algunos* commits de una rama, no todos. **git cherry-pick** copia exactamente los commits que nombres a donde estás.

La rama **experiments** tiene dos arreglos útiles (C2 y C4) y una prueba rara (C3) que no quieres.

ANTES



DESPUÉS



COMANDOS

```
$ git cherry-pick C2 C4
```

En la vida real: Uso típico: un bug se arregla en una rama de desarrollo y necesitas solo ese arreglo en la versión ya publicada.

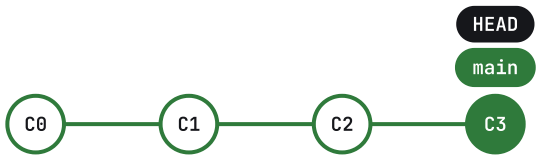
NIVEL 9 / 10

Tags: marcar versiones

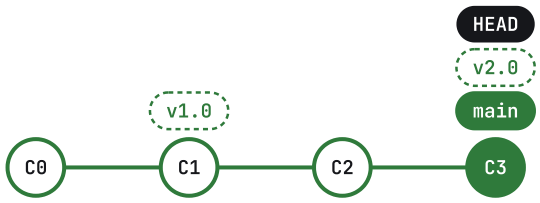
Un **tag** es una notita que nunca se mueve. Las ramas avanzan con cada commit; los tags se quedan quietos para siempre.

Por eso los tags se usan para versiones: **v1.0** siempre significará exactamente esa foto.

ANTES



DESPUÉS



COMANDOS

```
$ git tag v1.0 C1
$ git tag v2.0
```

En la vida real: GitHub convierte los tags en Releases, donde puedes adjuntar archivos (como el PDF de esta guía). Se suben con `git push --tags`.

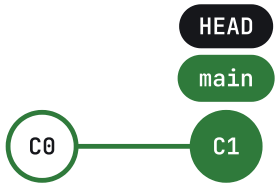
NIVEL 10 / 10

Desafío final: un flujo real

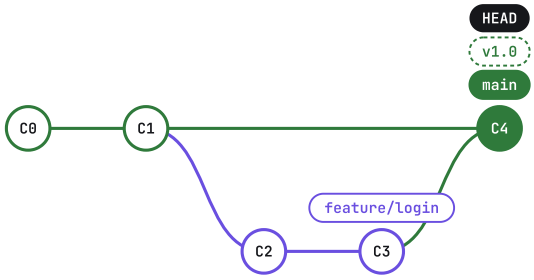
Junta todo, como trabajan los equipos todos los días:

1. Crea **feature/login** y haz 2 commits ahí.
2. Vuelve a **main** y haz merge con `--no-ff`, para que la historia muestre que se unió una función (aunque se pudiera hacer fast-forward).
3. Marca el resultado como **v1.0**.

ANTES



DESPUÉS



COMANDOS

```
$ git switch -c feature/login
$ git commit -m "Formulario de login"
$ git commit -m "Validación de login"
$ git switch main
$ git merge --no-ff feature/login
$ git tag v1.0
```

En la vida real: Así mismo se construyó este proyecto en GitHub: una rama por función, un pull request para cada una, merge a main y luego un tag de versión.

Hoja de referencia

`git commit -m "msj"`	Guardar una foto
`git switch -c nombre`	Crear una rama y moverte a ella
`git switch nombre`	Moverte a una rama
`git merge nombre`	Unir otra rama a la tuya
`git rebase nombre`	Repetir tus commits encima de otra rama
`git checkout C1`	Visitar un commit viejo (HEAD separado)
`main~2 / HEAD^`	Dos commits atrás / el padre
`git reset HEAD~1`	Mover la rama atrás (solo trabajo privado)
`git revert C3`	Agregar un commit que deshace C3 (seguro si es compartido)
`git cherry-pick C2 C4`	Copiar solo esos commits aquí
`git tag v1.0`	Marcar una versión (los tags no se mueven)
`git log / git status`	Ver historia / dónde estás